LinkKeeper — Complete Solution

> The whole MCP server and its client, every file — nothing to copy from elsewhere. Use it to **check your work**, not to skip the thinking. Build from the story first; read the hints if you get stuck. All of this runs against the real `mcp` SDK with no API key.

The final layout

```
<repo folder>/
├── src/
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py
│   ├── __init__.py
│   ├── config.py
│   ├── db.py
│   └── exceptions.py
├── tests/
│   ├── integration/
│   │   └── test_server.py
│   └── unit/
│       ├── test_config.py
│       └── test_db.py
├── .env.example
├── .gitignore
├── LICENSE
├── client.py
├── requirements.txt
└── server.py
```

Setup files

`requirements.txt`

```
mcp
python-dotenv
```

`.env.example`

```
LINKKEEPER_DB=linkkeeper.sqlite
LOG_LEVEL=INFO
```

`.gitignore`

```
.env
.venv/
__pycache__/
*.pyc
*.sqlite
```

Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text. (No `OPENAI_API_KEY` — this server has no LLM of its own; the *client's* agent brings the model.)

Settings, errors, logging

```
# src/exceptions.py
# The server's named errors. EmptyBookmarkError: a bookmark was given no
# URL. MissingConfigError: a required setting is missing.
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass


class EmptyBookmarkError(Exception):
    """A bookmark was submitted with an empty URL."""
    pass
```

```
# src/config.py
# Settings from .env into a checked Config. Both have safe defaults, so the
# server runs out of the box.
import os
from dataclasses import dataclass

from dotenv import load_dotenv


@dataclass
class Config:
    db_path: str
    log_level: str
```

```
def load_config() -> Config:
    load_dotenv()
    return Config(
        db_path=os.getenv("LINKKEEPER_DB", "linkkeeper.sqlite"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
    )
```

```
# src/utils/logger.py
# One logging setup. Logs to STDERR on purpose: for an MCP stdio server,
# stdout is the protocol channel, so a stray print there corrupts it.
import logging
import os
import sys


def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler(sys.stderr)
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

The SQLite store

```
# src/db.py
# The bookmarks store, backed by SQLite so bookmarks survive a restart. The
# tools in server.py call these functions; nothing else touches SQL.
import sqlite3

from src.exceptions import EmptyBookmarkError


def connect(db_path: str):
    connection = sqlite3.connect(db_path)
    connection.row_factory = sqlite3.Row
    return connection
```

```
def init_db(db_path: str) -> None:
    connection = connect(db_path)
    connection.execute(
        "CREATE TABLE IF NOT EXISTS bookmarks ("
        "id INTEGER PRIMARY KEY AUTOINCREMENT, "
        "url TEXT, title TEXT, tag TEXT)"
    )
    connection.commit()
    connection.close()


def add_bookmark(db_path: str, url: str, title: str, tag: str) -> None:
    if not url.strip():
        raise EmptyBookmarkError("url cannot be empty")
    connection = connect(db_path)
    connection.execute(
        "INSERT INTO bookmarks (url, title, tag) VALUES (?, ?, ?)",
        (url, title, tag),
    )
    connection.commit()
    connection.close()


def search_bookmarks(db_path: str, query: str) -> list:
    connection = connect(db_path)
    rows = connection.execute("SELECT * FROM bookmarks").fetchall()
    connection.close()
    hits = []
    for row in rows:
        blob = (row["title"] + " " + row["tag"] + " " + row["url"]).lower()
        if query.lower() in blob:
            hits.append(f"{row['title']} ({row['tag']}): {row['url']}")
    return hits


def all_bookmarks(db_path: str) -> list:
    connection = connect(db_path)
    rows = connection.execute("SELECT * FROM bookmarks").fetchall()
    connection.close()
    lines = []
    for row in rows:
        lines.append(f"{row['title']} ({row['tag']}): {row['url']}")
    return lines
```

The server (the three building blocks)

```
# server.py
# The MCP server: two Tools, one Resource, one Prompt, over stdio. Run it
# with a client, e.g.  mcp dev server.py  or  python client.py.
# IMPORTANT: never print() to stdout here — stdout is the protocol channel.
# All logging goes to stderr (see src/utils/logger.py).
import sys

from mcp.server.mcpserver import MCPServer

from src.config import load_config
from src.db import add_bookmark as db_add
from src.db import all_bookmarks as db_all
from src.db import init_db
from src.db import search_bookmarks as db_search
from src.exceptions import EmptyBookmarkError

CONFIG = load_config()
init_db(CONFIG.db_path)

mcp = MCPServer("linkkeeper-server")


@mcp.tool()
def add_bookmark(url: str, title: str, tag: str = "general") -> str:
    """Save a bookmark. Returns a confirmation, or an error if the url is
    empty. Use the tag to group related links."""
    try:
        db_add(CONFIG.db_path, url, title, tag)
    except EmptyBookmarkError as e:
        # Returning a clear "Error: ..." string (not raising) gives the
        # client a readable message instead of a generic tool failure.
        return f"Error: {e}"
    return f"Saved: {title} [{tag}]"


@mcp.tool()
def search_bookmarks(query: str) -> str:
    """Search saved bookmarks by a keyword in the title, tag, or url."""
    hits = db_search(CONFIG.db_path, query)
    if not hits:
        return "No matching bookmarks."
    return "\n".join(hits)
```

```
@mcp.resource("bookmarks://all")
def all_bookmarks_resource() -> str:
    """Every saved bookmark, as text."""
    rows = db_all(CONFIG.db_path)
    if not rows:
        return "No bookmarks yet."
    return "\n".join(rows)


@mcp.prompt()
def reading_list() -> str:
    """A reusable prompt that turns the saved bookmarks into a reading list."""
    return (
        "Using the bookmarks resource, group the saved links into a short, "
        "themed reading list with a one-line note on each."
    )


if __name__ == "__main__":
    print("linkkeeper server starting", file=sys.stderr)
    mcp.run(transport="stdio")
```

The client (discovers and uses all three)

```
# client.py
# A client that launches server.py over stdio, DISCOVERS what it offers
# (never importing from it), then uses all three building blocks. The
# empty-url call shows the clean, readable rejection.
import asyncio

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client


async def main() -> None:
    params = StdioServerParameters(command="python", args=["server.py"])
    try:
        async with stdio_client(params) as (read, write):
            async with ClientSession(read, write) as session:
                await session.initialize()

                tools = await session.list_tools()
                print("tools:", [t.name for t in tools.tools])
                resources = await session.list_resources()
                print("resources:", [str(r.uri) for r in resources.resources])
```

```
                prompts = await session.list_prompts()
                print("prompts:", [p.name for p in prompts.prompts])

                added = await session.call_tool(
                    "add_bookmark",
                    {"url": "https://docs.python.org", "title": "Py Docs",
                     "tag": "ref"},
                )
                print("add_bookmark:", added.content[0].text)

                bad = await session.call_tool(
                    "add_bookmark", {"url": "   ", "title": "oops"}
                )
                if bad.content[0].text.startswith("Error"):
                    print("empty url rejected cleanly:", bad.content[0].text)

                found = await session.call_tool(
                    "search_bookmarks", {"query": "py"}
                )
                print("search_bookmarks:", found.content[0].text)

                res = await session.read_resource("bookmarks://all")
                print("bookmarks://all:", res.contents[0].text)

                prompt = await session.get_prompt("reading_list")
                print("reading_list:", prompt.messages[0].content.text[:50])
    except Exception as e:
        print(f"Could not talk to the server: {e}")


if __name__ == "__main__":
    asyncio.run(main())
```

Expected output (`python client.py` , after `pip install mcp`):

```
tools: ['add_bookmark', 'search_bookmarks']
resources: ['bookmarks://all']
prompts: ['reading_list']
add_bookmark: Saved: Py Docs [ref]
empty url rejected cleanly: Error: url cannot be empty
search_bookmarks: Py Docs (ref): https://docs.python.org
bookmarks://all: Py Docs (ref): https://docs.python.org
reading_list: Using the bookmarks resource, group the saved lin
```

> **Two real gotchas, both confirmed by running it:** (1) the server must log to **stderr**, never `print()` to stdout — stdout carries the MCP protocol, and a stray print there breaks the client's parsing. (2) In the current `mcp` SDK a `CallToolResult`'s flag is `result.is_error` (snake_case). This client checks the returned **text** instead (`.startswith("Error")`), which is version-independent because the server returns a clear `"Error: ..."` string for a rejected bookmark.

The tests

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config


def test_default_db_path() -> None:
    os.environ.pop("LINKKEEPER_DB", None)
    config = load_config()
    result = "OK"
    if config.db_path != "linkkeeper.sqlite":
        result = "WRONG"
    print(f"case 1 (default db path)   -> {result}")


def test_env_overrides_db_path() -> None:
    os.environ["LINKKEEPER_DB"] = "custom.sqlite"
    config = load_config()
    result = "OK"
    if config.db_path != "custom.sqlite":
        result = "WRONG"
    os.environ.pop("LINKKEEPER_DB", None)
    print(f"case 2 (env overrides path) -> {result}")


if __name__ == "__main__":
    test_default_db_path()
    test_env_overrides_db_path()
```

```
# tests/unit/test_db.py
# The store directly, no MCP, no network. Each test uses its own fresh file.
# Run from the repo root: python -m tests.unit.test_db
```

```
import os

from src.db import add_bookmark, all_bookmarks, init_db, search_bookmarks
from src.exceptions import EmptyBookmarkError

DB = "test_linkkeeper.sqlite"


def fresh():
    if os.path.exists(DB):
        os.remove(DB)
    init_db(DB)


def test_add_and_search() -> None:
    fresh()
    add_bookmark(DB, "https://docs.python.org", "Py Docs", "ref")
    hits = search_bookmarks(DB, "py")
    result = "OK"
    if len(hits) != 1 or "Py Docs" not in hits[0]:
        result = "WRONG"
    print(f"case 1 (add then find)        -> {result}")


def test_empty_url_raises() -> None:
    fresh()
    result = "WRONG"
    try:
        add_bookmark(DB, "   ", "oops", "general")
    except EmptyBookmarkError:
        result = "OK"
    print(f"case 2 (empty url raises)     -> {result}")


def test_all_lists_every_row() -> None:
    fresh()
    add_bookmark(DB, "https://a.com", "A", "x")
    add_bookmark(DB, "https://b.com", "B", "y")
    rows = all_bookmarks(DB)
    result = "OK"
    if len(rows) != 2:
        result = "WRONG"
    print(f"case 3 (all lists every row)  -> {result}")


def test_persists_across_connections() -> None:
    fresh()
```

```
    add_bookmark(DB, "https://keep.me", "Keeper", "ref")
    # a new call = a new connection = proof it's really on disk
    result = "OK"
    if len(search_bookmarks(DB, "keeper")) != 1:
        result = "WRONG"
    print(f"case 4 (persists on disk)     -> {result}")


if __name__ == "__main__":
    test_add_and_search()
    test_empty_url_raises()
    test_all_lists_every_row()
    test_persists_across_connections()
```

```
# tests/integration/test_server.py
# Launches the real server over MCP and uses all three building blocks.
# Each case prints OK or WRONG. Run from the repo root:
#   python -m tests.integration.test_server
import asyncio
import os

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

TEST_DB = "it_linkkeeper.sqlite"


async def main() -> None:
    if os.path.exists(TEST_DB):
        os.remove(TEST_DB)
    os.environ["LINKKEEPER_DB"] = TEST_DB
    params = StdioServerParameters(command="python", args=["server.py"])
    async with stdio_client(params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            tool_names = [t.name for t in (await session.list_tools()).tools]
            r1 = "OK" if "add_bookmark" in tool_names else "WRONG"
            print(f"case 1 (tools discovered)   -> {r1}")

            added = await session.call_tool(
                "add_bookmark",
                {"url": "https://x.com", "title": "X", "tag": "ref"},
            )
            r2 = "OK" if "Saved" in added.content[0].text else "WRONG"
            print(f"case 2 (add works)          -> {r2}")
```

```
            bad = await session.call_tool(
                "add_bookmark", {"url": "  ", "title": "oops"}
            )
            r3 = "OK" if bad.content[0].text.startswith("Error") else "WRONG"
            print(f"case 3 (empty url -> Error)  -> {r3}")

            res = await session.read_resource("bookmarks://all")
            r4 = "OK" if "X" in res.contents[0].text else "WRONG"
            print(f"case 4 (resource read)      -> {r4}")

            prompt = await session.get_prompt("reading_list")
            text = prompt.messages[0].content.text
            r5 = "OK" if "reading list" in text.lower() else "WRONG"
            print(f"case 5 (prompt fetched)     -> {r5}")


if __name__ == "__main__":
    asyncio.run(main())
```

Expected unit output (all `OK`):

```
case 1 (default db path)   -> OK
case 2 (env overrides path) -> OK
```

```
case 1 (add then find)        -> OK
case 2 (empty url raises)     -> OK
case 3 (all lists every row)  -> OK
case 4 (persists on disk)     -> OK
```

Expected integration output (launches the real server):

```
case 1 (tools discovered)   -> OK
case 2 (add works)          -> OK
case 3 (empty url -> Error)  -> OK
case 4 (resource read)      -> OK
case 5 (prompt fetched)     -> OK
```

That's the whole server. If a client can discover your two tools, the resource, and the prompt, use all three, get a readable `"Error:"` on an empty URL, and find its bookmarks still there after a restart — you've built a real MCP server other agents can plug into.